КУРС И ПАМЯТКА

XSD с нуля

Шаг за шагом собираем схему заказа интернет-магазина: от первого тега до схемы из нескольких файлов. По дороге разбираем все конструкции, которые проверяет наш валидатор.

Как работать с курсом

Что понадобится и как проверять себя

Часть I. XML

1. Документ XML: элементы, атрибуты, текст
2. Пространства имён и префиксы

Часть II. Первая схема и типы

3. Что такое XSD. Первая схема
4. Встроенные типы
5. Свой простой тип: ограничения строк
6. Ограничения чисел
7. Наследование, анонимные типы, list, union

Часть III. Структура документа

8. Сложные типы: дочерние элементы и атрибуты
9. choice и all
10. Текст + атрибуты, наследование сложных типов
11. Повторное использование: ref, group, substitutionGroup
12. «Всеядные» конструкции: any, anyType
13. Схема из нескольких файлов

Часть IV. Наш валидатор

Этап 1 и этап 2: что ловит каждый
Три правила на примерах
Как код обходит схему

Памятка

Синтаксис, типы, фасеты, конструкции, частые ошибки — на нескольких страницах

Все примеры в курсе — настоящие файлы. Каждый XSD скомпилирован нашим `XsdSchemaCreator`, каждый XML проверен валидатором, а сообщения об ошибках взяты из этих проверок как есть.

Как работать с курсом

Курс построен как одна сквозная задача: мы описываем XML-документ «заказ» и шаг за шагом пишем для него схему. На каждом шаге появляется одно новое понятие.

Как устроен каждый шаг

1. **Объяснение**: что это за конструкция и какую проблему она решает.
2. **Схема** (`.xsd`) с новой конструкцией.
3. **Правильный XML**, который проходит проверку по этой схеме (зелёная рамка).
4. **Сломанный XML** (красная рамка): строка с ошибкой подсвечена, ниже настоящее сообщение валидатора и перевод на человеческий.
5. **Что скажет наш валидатор**: проходит ли схема наши три правила.

Как проверять себя

Лучше всего повторять шаги руками: создать файлы, вставить код, а потом ломать XML и смотреть на ошибки. Есть два удобных способа.

Способ 1. IntelliJ IDEA подсвечивает ошибки сама

Положите `.xsd` и `.xml` в одну папку и добавьте в корневой тег XML подсказку, где лежит схема:

order.xml

```xml
<order xmlns="urn:shop:order"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="urn:shop:order order.xsd">
    ...
</order>
```

`xsi:schemaLocation` — это пары «пространство имён — файл схемы» через пробел. Валидатору в нашем приложении эта подсказка не нужна (схему мы передаём ему сами), но IDE по ней поймёт, какой схемой проверять файл, и подчеркнёт ошибки прямо в редакторе.

Способ 2. Нашим кодом

Проверка.java

```java
XsdSchema xsd = new XsdSchemaCreator().createByFilePath("/абсолютный/путь/order.xsd"); // этап 1
new XsdSchemaValidator().validate(xsd);                                                 // этап 2
xsd.schema().newValidator().validate(new StreamSource(new File("/путь/order.xml")));   // проверка XML
```

Первые две строки проверяют **схему**, третья — **XML-документ** по этой схеме. Об этой разнице мы подробно поговорим в части IV.

ГЛАВНАЯ МЫСЛЬ КУРСА

XML — это данные. XSD — это **правила** для данных: какие теги можно писать, в каком порядке и какие значения в них допустимы. Наш валидатор — это ещё один уровень: **правила для правил**. Он следит, чтобы схема не разрешала слишком много.

ЧАСТЬ I

XML

Прежде чем описывать правила для XML, нужно уверенно читать сам XML: из чего состоит документ и как работают пространства имён. Без второго не понять, почему в схеме везде пишут `xs:`.

1 Документ XML

> Цель шага: знать, из чего состоит XML-документ, и уметь отличить правильно оформленный документ от сломанного.

XML — текстовый формат для хранения данных в виде **дерева**. Вот заказ в XML:

ok-order.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Это комментарий: парсер его пропускает -->
<order id="42" priority="high">
    <number>ORD-000042</number>
    <customer>
        <name>Иван Петров</name>
        <phone>+79991234567</phone>
    </customer>
    <comment>Позвонить за час &amp;amp; не опаздывать</comment>
    <gift/>
</order>
```

Из чего он состоит

Понятие	Пример	Что это
Пролог	`<?xml version="1.0" encoding="UTF-8"?>`	Необязательная первая строка: версия XML и кодировка файла.
Тег	`<number>`, `</number>`	Метка в угловых скобках. Открывающий тег начинает элемент, закрывающий (со слешем) заканчивает.
Элемент	`<number>ORD-000042</number>`	Открывающий тег + содержимое + закрывающий тег. Основная единица данных.
Пустой элемент	`<gift/>`	Элемент без содержимого. То же самое, что `<gift></gift>`.
Текстовое содержимое	`ORD-000042`	Текст между тегами — значение элемента.
Атрибут	`id="42"`	Пара «имя = значение» внутри открывающего тега. Значение всегда в кавычках.
Корневой элемент	`<order>`	Самый внешний элемент. В документе он ровно один.
Дочерний элемент	`<name>` внутри `<customer>`	Элемент, вложенный в другой (родительский).
Комментарий	`<!-- ... -->`	Пояснение для человека, парсер его пропускает.

Дерево этого документа:

```
order  (атрибуты: id, priority)
├── number      → текст «ORD-000042»
├── customer
│   ├── name    → текст «Иван Петров»
│   └── phone   → текст «+79991234567»
├── comment     → текст «Позвонить за час & не опаздывать»
└── gift        (пустой)
```

Элемент или атрибут?

Одни и те же данные можно записать по-разному: `<order id="42">` или `<order><id>42</id></order>`. Разница в возможностях:

	Атрибут	Элемент
Значение	только строка	текст или другие элементы внутри
Повторения	каждый атрибут максимум один раз на элементе	может повторяться (несколько `<item>`)
Порядок	не важен	может быть важен
Обычно используют для	метаданных: id, код, валюта, флаг	самих данных и всего, у чего есть структура

Спецсимволы

Символы `<` и `&` в тексте XML писать нельзя: парсер примет их за начало тега или ссылки. Вместо них пишут **сущности**:

Символ	Как писать	Когда обязательно
`<`	`<`	всегда
`&`	`&`	всегда

Символ	Как писать	Когда обязательно
`>`	`>`	желательно
`"`	`"`	внутри значения атрибута в двойных кавычках
`'`	`'`	внутри значения атрибута в одинарных кавычках

Правильно оформленный документ

Документ, который парсер вообще может прочитать, называется **правильно оформленным** (well-formed). Требования простые:

1. Ровно один корневой элемент.
2. Каждый открытый тег закрыт, а вложенность не перекрывается: `<a><b></b></a>` можно, `<a><b></a></b>` нельзя.
3. Значения атрибутов в кавычках.
4. Регистр важен: `<Order>` и `<order>` — разные теги.
5. `<` и `&` в тексте только через сущности.

Нарушение любого пункта — фатальная ошибка: дальше парсер документ не читает. Схема на этом уровне ещё не участвует.

ПОПРОБУЙТЕ СЛОМАТЬ

Ниже четыре сломанных документа и то, что про них говорит парсер.

`bad-unclosed.xml` — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <order>
3     <number>ORD-000042</order>
```

Сообщение валидатора (строка 3)

```
The element type "number" must be terminated by the matching end-tag "</number>".
```

Тег `<number>` открыт, но вместо его закрытия идёт `</order>`.

`bad-two-roots.xml` — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <order/>
3 <order/>
```

Сообщение валидатора (строка 3)

```
The markup in the document following the root element must be well-formed.
```

После корневого элемента ничего, кроме комментариев, быть не может. Второй корень запрещён.

`bad-attr-quotes.xml` — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <order id=42/>
```

Сообщение валидатора (строка 2)

```
Open quote is expected for attribute "id" associated with an element type "order".
```

Значение атрибута без кавычек.

`bad-special-char.xml` — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <comment>Цена < 100</comment>
```

Сообщение валидатора (строка 2)

```
The content of elements must consist of well-formed character data or markup.
```

Символ < в тексте парсер принял за начало тега. Нужно писать `<`.

2 Пространства имён

> Цель шага: понять, что такое `xmlns`, префикс и пространство имён по умолчанию. Это ключ к пониманию, почему в схеме пишут `xs:string` и почему наш валидатор смотрит не на префикс, а на то, что за ним стоит.

Проблема: одинаковые имена

Представьте, что в заказе встречаются `<price>` из каталога магазина и `<price>` из модуля доставки. Имя одно, а смысл и правила разные. Чтобы различать такие элементы, в XML есть **пространства имён** (namespaces).

Как это работает

Пространство имён — это просто **уникальная строка**, обычно похожая на адрес: `urn:shop:order` или `http://example.com/order`. Это не ссылка: никто по ней ничего не скачивает, это только имя. Писать её целиком у каждого тега неудобно, поэтому объявляют **префикс** — короткий псевдоним:

ok-prefix.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<o:order xmlns:o="urn:shop:order"
         xmlns:c="urn:shop:common">
    <o:number>ORD-000042</o:number>
    <c:price currency="RUB">1500.00</c:price>
</o:order>
```

- `xmlns:o="urn:shop:order"` — объявление: «префикс `o` означает `urn:shop:order`».
- `<o:order>` — элемент `order` из пространства `urn:shop:order`.
- `<c:price>` — элемент `price` из пространства `urn:shop:common`.

САМОЕ ВАЖНОЕ В ЭТОМ ШАГЕ

Настоящее имя элемента — это пара **(пространство имён, локальное имя)**, например `(urn:shop:order, order)`. Префикс — только сокращение, и он может быть любым. `<o:order xmlns:o="urn:shop:order">` и `<shop:order xmlns:shop="urn:shop:order">` — **один и тот же** элемент.

Пространство имён по умолчанию

Объявление без префикса, `xmlns="..."`, действует на все элементы **без префикса**. Этот документ полностью равнозначен предыдущему:

ok-default.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order"
       xmlns:c="urn:shop:common">
    <number>ORD-000042</number>
    <c:price currency="RUB">1500.00</c:price>
</order>
```

Именно так мы будем писать XML дальше в курсе: основное пространство по умолчанию, остальные через префиксы.

Ещё три правила

- **Область действия.** Объявление действует на тот элемент, где написано, и на всех его потомков.
- **Атрибуты без префикса не попадают в пространство по умолчанию.** `currency="RUB"` просто принадлежит своему элементу. Поэтому атрибуты мы пишем без префиксов.
- **Префикс нужно объявить.** Иначе парсер не знает, что он означает, и документ сломан:

bad-unbound-prefix.xml — ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<o:order>
    <o:number>ORD-000042</o:number>
</o:order>
```

Сообщение валидатора (строка 2)

```
The prefix "o" for element "o:order" is not bound.
```

Префикс o используется, но нигде не объявлен через `xmlns:o="..."`.

Префиксы встречаются и в значениях

Запись вида `префикс:имя` называется **QName** (qualified name — полное имя). Она бывает не только в названиях тегов, но и **внутри значений атрибутов**. В схемах это сплошь и рядом: `type="xs:string"`. Работает тот же механизм: чтобы понять, что такое `xs:string`, нужно найти объявление `xmlns:xs="..."` и посмотреть, какое пространство за ним стоит.

КАК ЭТО ИСПОЛЬЗУЕТ НАШ ВАЛИДАТОР

Метод `builtInTypeName` делит `xs:string` на префикс `xs` и имя `string`, ищет объявление префикса (`lookupNamespaceURI`) и проверяет, что за ним стоит пространство XML Schema. Поэтому он одинаково понимает `xs:string`, `xsd:string` и просто `string`, если XML Schema объявлена пространством по умолчанию.

ЧАСТЬ II

Первая схема и типы

Пишем первую XSD и учимся описывать, какие **значения** допустимы: числа, даты, строки по шаблону, списки значений. Всё это — простые типы.

3 Что такое XSD. Первая схема

> Цель шага: понять, как устроен файл схемы, что значат атрибуты `xs:schema`, и объявить первый элемент.

XSD (XML Schema Definition) — файл с правилами для XML-документов: какие элементы и атрибуты в них могут быть, в каком порядке и с какими значениями. Проверка документа по схеме называется **валидацией**.

Важный момент: **XSD сам является XML-документом**. У него есть пролог, корневой элемент, теги и атрибуты. Поэтому его можно прочитать обычным XML-парсером, и наш валидатор именно так и делает.

Первая схема

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:element name="note" type="xs:string"/>

</xs:schema>
```

Разбираем по строкам

Что написано	Что это значит
`<xs:schema>`	Корневой элемент любой схемы.
`xmlns:xs="http://www.w3.org/2001/XMLSchema"`	Объявляем префикс `xs` для пространства имён XML Schema. Все теги языка схем (`xs:element`, `xs:string` …) живут в этом пространстве. Префикс `xs` — традиция, встречается и `xsd`: по шагу 2 это одно и то же.
`targetNamespace="urn:shop:order"`	В какое пространство имён попадут элементы, которые объявляет эта схема. XML-документ должен использовать именно его.
`elementFormDefault="qualified"`	Вложенные (локальные) элементы тоже принадлежат `targetNamespace`. Без этого вложенные элементы в XML пришлось бы писать **без** пространства имён — частая ловушка. Ставьте `qualified` всегда.
`<xs:element name="note" type="xs:string"/>`	**Объявление** элемента: в документе может быть элемент `note`, его значение — строка.

Объявление и использование

Две вещи, которые легко перепутать:

- **Объявить** элемент — описать правило в XSD: `<xs:element name="note" .../>`.
- **Использовать** элемент — написать его в XML: `<note>...</note>`.

Объявление, которое лежит прямо внутри `xs:schema`, называется **глобальным**. Только глобальный элемент может быть корнем XML-документа.

ok-note.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<note xmlns="urn:shop:order">Привет, XSD!</note>
```

Элемент `note` записан в пространстве `urn:shop:order` (через `xmlns` по умолчанию). Это и есть `targetNamespace` схемы, поэтому валидатор находит объявление.

ПОПРОБУЙТЕ СЛОМАТЬ

bad-no-namespace.xml — ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<note>Привет, XSD!</note>
```

Сообщение валидатора (строка 2)

```
cvc-elt.1.a: Cannot find the declaration of element 'note'.
```

Элемент `note` без пространства имён — это **другой** элемент, не тот, что объявлен в схеме (там `note` из `urn:shop:order`). Объявления для него нет.

bad-unknown-element.xml ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <message xmlns="urn:shop:order">Привет</message>
```

Сообщение валидатора (строка 2)

`cvc-elt.1.a: Cannot find the declaration of element 'message'.`

Элемент `message` в схеме не объявлен.

Что скажет наш валидатор

Наш XsdSchemaValidator отклонит эту схему:

`[schema.xsd]: <xs:element name="note"> type="xs:string": для строкового типа должно быть задано ограничение (xs:length, xs:maxLength, xs:enumeration или xs:pattern) через xs:restriction`

Пока мы учимся, схемы шагов 3 и 4 нарочно используют встроенные типы «как есть». Начиная с шага 5 мы будем писать схемы, которые проходят наши правила.

4 Встроенные типы

> Цель шага: знать основные встроенные типы, как они связаны между собой и почему правила нашего валидатора касаются именно `xs:string` и семейства `xs:decimal`.

Тип отвечает на вопрос «какие значения допустимы». В XML Schema около 45 встроенных типов. Все они живут в пространстве имён XML Schema, поэтому пишутся с префиксом: `xs:date`, `xs:int`.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:element name="createdAt" type="xs:date"/>     <!-- дата: ГГГГ-ММ-ДД -->
    <xs:element name="paid"      type="xs:boolean"/>  <!-- true / false / 1 / 0 -->
    <xs:element name="total"     type="xs:decimal"/>  <!-- число с дробной частью -->
    <xs:element name="count"     type="xs:int"/>      <!-- целое от -2147483648 до 2147483647 -->

</xs:schema>
```

Основные встроенные типы

Тип	Что хранит	Подходит	Не подходит
`xs:string`	любой текст любой длины	`Привет`, пустая строка	— (подходит всё)
`xs:boolean`	логическое значение	`true` `false` `1` `0`	`yes`, `TRUE`
`xs:date`	дата	`2026-11-03`	`03.11.2026`
`xs:dateTime`	дата и время	`2026-11-03T10:15:00`	`2026-11-03 10:15`
`xs:decimal`	число, можно с дробной частью, **любой величины**	`1500.50` `-3`	`1 500,50`
`xs:integer`	целое **любой величины**	`42` `-7`	`4.2`
`xs:positiveInteger`	целое ≥ 1	`1`	`0`
`xs:nonNegativeInteger`	целое ≥ 0	`0`	`-1`
`xs:negativeInteger`	целое ≤ −1	`-1`	`0`
`xs:nonPositiveInteger`	целое ≤ 0	`0`	`1`
`xs:long`, `xs:int`	целое в фиксированных границах (как в Java)	`2147483647` для int	`3000000000` для int

Типы образуют дерево

Каждый встроенный тип получен из другого **сужением**: `xs:integer` — это `xs:decimal` без дробной части, `xs:int` — это `xs:long` в меньших границах. На вершине — два особых типа, к которым мы вернёмся в шаге 12.

```
xs:anyType                    что угодно: любой текст, любые элементы, любые атрибуты
└── xs:anySimpleType          любой текст
    ├── xs:string             любая строка
    │   └── xs:normalizedString → xs:token → ...
    ├── xs:decimal            любое число
    │   └── xs:integer        любое целое
    │       ├── xs:nonNegativeInteger (≥0) → xs:positiveInteger (≥1)
    │       ├── xs:nonPositiveInteger (≤0) → xs:negativeInteger (≤−1)
    │       └── xs:long → xs:int → xs:short → xs:byte     (уже с границами)
    ├── xs:boolean
    ├── xs:date, xs:dateTime, xs:time, ...
    └── ...
```

ОТКУДА ВЗЯЛИСЬ НАШИ ПРАВИЛА

Посмотрите на выделенные синим типы: у них **нет верхнего предела**. `xs:string` допускает строку в гигабайт, `xs:decimal` и `xs:integer` — число с тысячей цифр, а у `xs:positiveInteger` и его соседей есть только одна граница. Отсюда **правило 1** (строки нужно ограничить) и **правило 2** (эти шесть числовых типов нужно ограничить). `xs:int` и `xs:long` уже ограничены по определению, поэтому правило их не касается. Красные типы на вершине — тема **правила 3**.

ok-date.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<createdAt xmlns="urn:shop:order">2026-11-03</createdAt>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-date-format.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <createdAt xmlns="urn:shop:order">03.11.2026</createdAt>
```

Сообщение валидатора (строка 2)

`cvc-datatype-valid.1.2.1: '03.11.2026' is not a valid value for 'date'.`

Дата в XML всегда пишется как ГГГГ-ММ-ДД.

bad-boolean.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <paid xmlns="urn:shop:order">yes</paid>
```

Сообщение валидатора (строка 2)

`cvc-datatype-valid.1.2.1: 'yes' is not a valid value for 'boolean'.`

Для `xs:boolean` допустимы только `true`, `false`, `1`, `0`.

bad-decimal.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <total xmlns="urn:shop:order">1 500,50</total>
```

Сообщение валидатора (строка 2)

`cvc-datatype-valid.1.2.1: '1 500,50' is not a valid value for 'decimal'.`

Ни пробелов, ни запятой: разделитель дробной части — точка.

bad-int-range.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <count xmlns="urn:shop:order">3000000000</count>
```

Сообщение валидатора (строка 2)

`cvc-maxInclusive-valid: Value '3000000000' is not facet-valid with respect to maxInclusive '2147483647' for type 'int'.`

У `xs:int` есть встроенная верхняя граница — та же, что у `int` в Java.

Наш XsdSchemaValidator отклонит эту схему:

```
[schema.xsd]: <xs:element name="total"> type="xs:decimal": для числового типа должно быть задано ограничение (xs:totalDigits или xs:min* + xs:max*) через xs:restriction
```

5 Свой простой тип: ограничения строк

> Цель шага: научиться сужать встроенный тип с помощью `xs:simpleType`, `xs:restriction` и фасетов. Это основа правил 1 и 2.

Встроенные типы слишком широкие: номер заказа не может быть произвольной строкой, а статус — любым словом. Поэтому делают **свой тип**: берут встроенный и сужают его.

Как устроен свой простой тип

```
<xs:simpleType name="ИмяТипа">              ← объявляем тип с именем
    <xs:restriction base="xs:string">      ← берём за основу (base) встроенный тип и сужаем
        <xs:maxLength value="100"/>        ← фасет: одно конкретное ограничение
        <xs:pattern value="..."/>          ← ещё фасет; действуют все сразу
    </xs:restriction>
</xs:simpleType>
```

Фасет (facet) — это один параметр ограничения: максимальная длина, шаблон, список допустимых значений. Внутри одного `xs:restriction` фасеты складываются: значение должно удовлетворять **всем** сразу.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <!-- Номер заказа: строго по шаблону -->
    <xs:simpleType name="OrderNumber">
        <xs:restriction base="xs:string">
            <xs:pattern value="ORD-[0-9]{6}"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Имя клиента: от 1 до 100 символов -->
    <xs:simpleType name="CustomerName">
        <xs:restriction base="xs:string">
            <xs:minLength value="1"/>
            <xs:maxLength value="100"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Статус: только одно из перечисленных значений -->
    <xs:simpleType name="Status">
        <xs:restriction base="xs:string">
            <xs:enumeration value="NEW"/>
            <xs:enumeration value="PAID"/>
            <xs:enumeration value="SHIPPED"/>
            <xs:enumeration value="CANCELLED"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Индекс: ровно 6 символов, и все — цифры -->
    <xs:simpleType name="Zip">
        <xs:restriction base="xs:string">
            <xs:length value="6"/>
            <xs:pattern value="[0-9]+"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:element name="number"       type="o:OrderNumber"/>
    <xs:element name="customerName" type="o:CustomerName"/>
    <xs:element name="status"       type="o:Status"/>
    <xs:element name="zip"          type="o:Zip"/>

</xs:schema>
```

Свои типы и префикс `o:`

Заметьте две новые вещи в `xs:schema` и в объявлениях элементов:

- `xmlns:o="urn:shop:order"` — префикс для **нашего** пространства имён, того же, что в `targetNamespace`. Всё, что мы объявляем в схеме (элементы и типы), попадает туда.
- `type="o:OrderNumber"` — ссылка на наш тип. Без префикса `o:` валидатор искал бы тип `OrderNumber` вне пространства имён и не нашёл бы.

Тип с именем (`name="..."`) называется **именованным**: объявили один раз — используем в любом количестве мест.

Фасеты для строк

Фасет	Что ограничивает	Пример
`length`	точная длина	`value="6"` : ровно 6 символов
`minLength`	минимальная длина	`value="1"` : не пустая
`maxLength`	максимальная длина	`value="100"` : не длиннее 100
`pattern`	регулярное выражение для всего значения	`value="ORD-[0-9]{6}"`
`enumeration`	одно значение из списка; каждый вариант — отдельный тег	`NEW` , `PAID` , ...
`whiteSpace`	что делать с пробелами: `preserve` , `replace` , `collapse`	не ограничивает содержимое

ДВА ИСКЛЮЧЕНИЯ ИЗ ПРАВИЛА «ВСЕ ФАСЕТЫ СРАЗУ»

Несколько `enumeration` — это **список вариантов** (подходит любой из них). Несколько `pattern` в одном `restriction` тоже работают как «или»: тип с шаблонами `[0-9]{4}` и `[A-Z]{2}` примет и `1234` , и `AB` , но не `A1` :

```
cvc-pattern-valid: Value 'A1' is not facet-valid with respect to pattern '[0-9]{4}|[A-Z]{2}' for type 'Code'.
```

Шаблоны (pattern) в двух словах

Шаблон всегда проверяет значение **целиком**: `[0-9]+` значит «только цифры», а не «есть цифры». Писать `^` и `$` не нужно.

Запись	Значение
`[0-9]` , `[a-z]` , `[A-Z]`	один символ из диапазона
`\d` , `\s`	цифра, пробельный символ
`[^@]`	любой символ, кроме `@`
`{6}` , `{2,10}`	ровно 6 раз; от 2 до 10 раз
`+` , `*` , `?`	1 и больше; 0 и больше; 0 или 1
`a|b`	или
`\+` , `\*` , `\.`	сами символы `+` , `*` , `.`

ok-number.xml — ✔ проходит проверку

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <number xmlns="urn:shop:order">ORD-000042</number>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-number-pattern.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <number xmlns="urn:shop:order">ORD-42</number>
```

Сообщение валидатора (строка 2)

```
cvc-pattern-valid: Value 'ORD-42' is not facet-valid with respect to pattern 'ORD-[0-9]{6}' for type 'OrderNumber'.
```

После `ORD-` должно идти ровно 6 цифр.

bad-status-enum.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <status xmlns="urn:shop:order">DONE</status>
```

Сообщение валидатора (строка 2)

```
cvc-enumeration-valid: Value 'DONE' is not facet-valid with respect to enumeration '[NEW, PAID, SHIPPED, CANCELLED]'. It must be a value from the enumeration.
```

`DONE` нет в списке допустимых значений.

bad-zip-length.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <zip xmlns="urn:shop:order">10100</zip>
```

Сообщение валидатора (строка 2)
`cvc-length-valid: Value '10100' with length = '5' is not facet-valid with respect to length '6' for type 'Zip'.`

`length="6"` требует ровно 6 символов.

`bad-zip-letters.xml` ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <zip xmlns="urn:shop:order">10100A</zip>
```

Сообщение валидатора (строка 2)
`cvc-pattern-valid: Value '10100A' is not facet-valid with respect to pattern '[0-9]+' for type 'Zip'.`

Длина правильная, но шаблон `[0-9]+` требует только цифры. Фасеты действуют одновременно.

`bad-name-empty.xml` ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <customerName xmlns="urn:shop:order"></customerName>
```

Сообщение валидатора (строка 2)
`cvc-minLength-valid: Value '' with length = '0' is not facet-valid with respect to minLength '1' for type 'CustomerName'.`

`minLength="1"` запрещает пустую строку.

Что скажет наш валидатор

Наш XsdSchemaValidator: схема проходит все три правила.

ПРАВИЛО 1

Строковый тип должен иметь хотя бы один из фасетов `length`, `maxLength`, `enumeration`, `pattern`. Каждый из них не даёт значению быть бесконечным или произвольным. А вот `minLength` и `whiteSpace` не в счёт: строка с `minLength="1"` всё ещё может быть сколь угодно длинной.

6 Ограничения чисел

> Цель шага: знать числовые фасеты и понимать, что требует правило 2.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <!-- Сумма: не больше 18 цифр всего, из них не больше 2 после точки -->
    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Количество: от 1 до 999 включительно -->
    <xs:simpleType name="Quantity">
        <xs:restriction base="xs:positiveInteger">
            <xs:minInclusive value="1"/>
            <xs:maxInclusive value="999"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Скидка в процентах: от 0 включительно до 100 НЕ включительно -->
    <xs:simpleType name="Discount">
        <xs:restriction base="xs:decimal">
            <xs:minInclusive value="0"/>
            <xs:maxExclusive value="100"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:element name="total"    type="o:Amount"/>
    <xs:element name="quantity" type="o:Quantity"/>
    <xs:element name="discount" type="o:Discount"/>

</xs:schema>
```

Фасеты для чисел

Фасет	Что ограничивает	Пример
`totalDigits`	сколько цифр всего (до и после точки)	`18` : не больше 18 цифр
`fractionDigits`	сколько цифр после точки	`2` : копейки
`minInclusive`	минимум, **включая** само значение (≥)	`1` : можно 1
`maxInclusive`	максимум, **включая** (≤)	`999` : можно 999
`minExclusive`	минимум, **не включая** (>)	`0` : строго больше нуля
`maxExclusive`	максимум, **не включая** (<)	`100` : 99.9 можно, 100 нельзя
`enumeration` , `pattern`	работают и для чисел	редко нужны

Какие фасеты к каким типам применимы

Фасет должен иметь смысл для базового типа: у числа нет «длины», у строки нет «количества цифр». Неподходящий фасет — ошибка уже на этапе компиляции схемы (этап 1).

Фасет	строки	decimal и integer-семейство	даты	list
`length` , `minLength` , `maxLength`	**да**	**нет**	**нет**	**да — число элементов списка**
`pattern` , `enumeration`	**да**	**да**	**да**	**да**
`totalDigits` , `fractionDigits`	**нет**	**да**	**нет**	**нет**
`min/maxInclusive` , `min/maxExclusive`	**нет**	**да**	**да**	**нет**
`whiteSpace`	**да**	только `collapse`	только `collapse`	только `collapse`

ok-discount.xml — **✔ проходит проверку**

```xml
<?xml version="1.0" encoding="UTF-8"?>
<discount xmlns="urn:shop:order">99.9</discount>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-total-fraction.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <total xmlns="urn:shop:order">10.555</total>
```

Сообщение валидатора (строка 2)

```
cvc-fractionDigits-valid: Value '10.555' has 3 fraction digits, but the number of fraction digits has been limited to 2.
```

Три знака после точки, а разрешено два.

bad-quantity-max.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <quantity xmlns="urn:shop:order">1000</quantity>
```

Сообщение валидатора (строка 2)

```
cvc-maxInclusive-valid: Value '1000' is not facet-valid with respect to maxInclusive '999' for type 'Quantity'.
```

Больше верхней границы `maxInclusive="999"`.

bad-quantity-zero.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <quantity xmlns="urn:shop:order">0</quantity>
```

Сообщение валидатора (строка 2)

```
cvc-minInclusive-valid: Value '0' is not facet-valid with respect to minInclusive '1' for type 'Quantity'.
```

Меньше нижней границы.

bad-discount-100.xml — ✘ ошибка

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <discount xmlns="urn:shop:order">100</discount>
```

Сообщение валидатора (строка 2)

```
cvc-maxExclusive-valid: Value '100' is not facet-valid with respect to maxExclusive '100.0' for type 'Discount'.
```

`maxExclusive="100"`: сама сотня уже не входит.

Наш XsdSchemaValidator: схема проходит все три правила.

ПРАВИЛО 2

Тип на основе `xs:decimal`, `xs:integer`, `xs:positiveInteger`, `xs:nonNegativeInteger`, `xs:negativeInteger` или `xs:nonPositiveInteger` должен иметь либо `totalDigits`, либо **обе** границы: одну из `minInclusive` / `minExclusive` и одну из `maxInclusive` / `maxExclusive`. Одна граница или только `fractionDigits` не в счёт: число всё ещё может быть сколь угодно большим.

7 Наследование, анонимные типы, list, union

> Цель шага: узнать ещё четыре способа строить простые типы. Все они встречаются в коде валидатора.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:simpleType name="CustomerName">
        <xs:restriction base="xs:string">
            <xs:minLength value="1"/>
            <xs:maxLength value="100"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Наследование: берём CustomerName и сужаем ещё сильнее -->
    <xs:simpleType name="ShortName">
        <xs:restriction base="o:CustomerName">
            <xs:maxLength value="30"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Один тег: 2-10 строчных латинских букв -->
    <xs:simpleType name="TagCode">
        <xs:restriction base="xs:string">
            <xs:pattern value="[a-z]{2,10}"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Список: несколько TagCode через пробел -->
    <xs:simpleType name="Tags">
        <xs:list itemType="o:TagCode"/>
    </xs:simpleType>

    <xs:simpleType name="Asap">
        <xs:restriction base="xs:string">
            <xs:enumeration value="ASAP"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Объединение: либо дата, либо слово ASAP -->
    <xs:simpleType name="DeliveryDate">
        <xs:union memberTypes="xs:date o:Asap"/>
    </xs:simpleType>

    <xs:element name="shortName"    type="o:ShortName"/>
    <xs:element name="tags"         type="o:Tags"/>
    <xs:element name="deliveryDate" type="o:DeliveryDate"/>

    <!-- Анонимный тип: описан прямо внутри элемента, без имени -->
    <xs:element name="comment">
        <xs:simpleType>
            <xs:restriction base="xs:string">
                <xs:maxLength value="500"/>
            </xs:restriction>
        </xs:simpleType>
    </xs:element>

</xs:schema>
```

1. Наследование: restriction от своего типа

В `base` можно указать не только встроенный тип, но и **свой**. Тогда новый тип получает все фасеты базового и добавляет свои. `ShortName` — это `CustomerName` (от 1 до 100 символов), сужённый до 30.

Сужать можно, расширять нельзя: `maxLength="200"` поверх базового `100` — ошибка компиляции схемы.

2. Анонимный тип

Элемент `comment` не ссылается на тип через `type`: тип описан **прямо внутри** элемента и у него нет имени. Такой тип называется **анонимным** (или вложенным). Он нужен, когда тип используется в одном месте и придумывать ему имя незачем. Переиспользовать его нельзя.

	Именованный тип	Анонимный тип
Где объявлен	прямо в `xs:schema`, с `name`	внутри `xs:element` или `xs:attribute`, без `name`

	Именованный тип	Анонимный тип
Как подключить	`type="o:ИмяТипа"`	никак, он уже на месте
Переиспользование	сколько угодно раз	только здесь

3. Список: `xs:list`

Значение — несколько значений через пробел, каждое проверяется по `itemType`: `<tags>gift urgent fragile</tags>`.

4. Объединение: `xs:union`

Значение подходит, если подходит под **любой** тип из `memberTypes` (имена через пробел). `DeliveryDate` принимает и дату, и слово `ASAP`.

ok-tags.xml ✔ проходит проверку

```xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <tags xmlns="urn:shop:order">gift urgent fragile</tags>
```

ok-delivery-asap.xml ✔ проходит проверку

```xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <deliveryDate xmlns="urn:shop:order">ASAP</deliveryDate>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-shortname-long.xml ✘ ошибка

```xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <shortName xmlns="urn:shop:order">Иван Иванович Петров-Водкин-Сидоров</shortName>
```

Сообщение валидатора (строка 2)

```
cvc-maxLength-valid: Value 'Иван Иванович Петров-Водкин-Сидоров' with length = '35' is not facet-valid with respect to maxLength '30' for type 'ShortName'.
```

35 символов. Для `CustomerName` это нормально, но `ShortName` сузил предел до 30.

bad-tags-upper.xml ✘ ошибка

```xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <tags xmlns="urn:shop:order">gift Urgent</tags>
```

Сообщение валидатора (строка 2)

```
cvc-pattern-valid: Value 'Urgent' is not facet-valid with respect to pattern '[a-z]{2,10}' for type 'TagCode'.
```

Второй элемент списка не подходит под `TagCode`: заглавная буква.

bad-delivery-tomorrow.xml ✘ ошибка

```xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <deliveryDate xmlns="urn:shop:order">tomorrow</deliveryDate>
```

Сообщение валидатора (строка 2)

```
cvc-datatype-valid.1.2.3: 'tomorrow' is not a valid value of union type 'DeliveryDate'.
```

Не дата и не `ASAP` — ни один тип объединения не подошёл.

Наш XsdSchemaValidator: схема проходит все три правила.

ЧТО ЗДЕСЬ ВАЖНО ДЛЯ НАШИХ ПРАВИЛ

- `restriction base="o:CustomerName"` наш валидатор **не проверяет**: база — свой тип, и он проверяется там, где объявлен. Если `CustomerName` ограничен, то и `ShortName` ограничен.
- `xs:list itemType="xs:string"` или `xs:union memberTypes="... xs:string"` — нарушение правила 1: в список или объединение попадает неограниченная строка. Правильно — указывать свои ограниченные типы, как `o:TagCode` и `o:Asap` здесь.
- `xs:date` в объединении — не нарушение: даты не входят в наши правила.

ЧАСТЬ III

Структура документа

Простые типы описывают значения. Теперь описываем **форму** документа: какие элементы вложены в какие, в каком порядке, сколько раз и с какими атрибутами. Это сложные типы.

8 Сложные типы: дочерние элементы и атрибуты

> Цель шага: собрать первый настоящий заказ: элемент с вложенными элементами, атрибутами и повторениями.

Простой и сложный тип

Главное различие в XML Schema:

	Простой тип (`xs:simpleType`)	Сложный тип (`xs:complexType`)
Что внутри	только текст-значение	дочерние элементы и/или атрибуты
Пример в XML	`<quantity>2</quantity>`	`<item sku="..."><name>...</name></item>`
Подходит для	элементов **и атрибутов**	только элементов

Как только у элемента появляется хотя бы один атрибут или дочерний элемент, ему нужен сложный тип. У атрибута тип всегда простой: внутри атрибута не может быть тегов.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <!-- Простые типы из прошлых шагов -->
    <xs:simpleType name="OrderNumber">
        <xs:restriction base="xs:string">
            <xs:pattern value="ORD-[0-9]{6}"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="ProductName">
        <xs:restriction base="xs:string">
            <xs:maxLength value="200"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Quantity">
        <xs:restriction base="xs:positiveInteger">
            <xs:minInclusive value="1"/>
            <xs:maxInclusive value="999"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Sku">
        <xs:restriction base="xs:string">
            <xs:pattern value="[A-Z]{3}-[0-9]{4}"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Comment">
        <xs:restriction base="xs:string">
            <xs:maxLength value="500"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- Сложный тип: позиция заказа -->
    <xs:complexType name="Item">
        <xs:sequence>                                        <!-- дочерние элементы строго по порядку -->
            <xs:element name="name"     type="o:ProductName"/>
            <xs:element name="quantity" type="o:Quantity"/>
            <xs:element name="price"    type="o:Amount"/>
        </xs:sequence>
        <xs:attribute name="sku"  type="o:Sku"     use="required"/>  <!-- обязательный -->
        <xs:attribute name="gift" type="xs:boolean" default="false"/> <!-- необязательный -->
    </xs:complexType>

    <!-- Корневой элемент заказа -->
    <xs:element name="order">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="number" type="o:OrderNumber"/>
                <xs:element name="items">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="item" type="o:Item"
                                        minOccurs="1" maxOccurs="unbounded"/>
```

```
                    </xs:sequence>
                </xs:complexType>
            </xs:element>
            <xs:element name="comment" type="o:Comment" minOccurs="0"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>

</xs:schema>
```

Как устроен complexType

```
<xs:complexType name="Item">
    <xs:sequence>                ← 1. модель содержимого: какие дочерние элементы и в каком порядке
        <xs:element name="name" .../>
        ...
    </xs:sequence>
    <xs:attribute name="sku" .../>  ← 2. атрибуты — всегда ПОСЛЕ модели содержимого
</xs:complexType>
```

`xs:sequence` — дочерние элементы **строго в указанном порядке**. Другие варианты, `xs:choice` и `xs:all`, будут в шаге 9.

Локальные и глобальные элементы

Элементы `name`, `quantity`, `price` объявлены **внутри** сложного типа. Это **локальные** объявления: они действуют только внутри этого типа. В разных типах может быть свой `name` со своим типом.

	Глобальный элемент	Локальный элемент
Где объявлен	прямо в `xs:schema`	внутри `xs:complexType` (в `sequence`, `choice`, `all`)
Может быть корнем XML	да	нет
Можно подключить через `ref`	да (шаг 11)	нет

Сколько раз: `minOccurs` и `maxOccurs`

Запись	Смысл
ничего не указано	ровно один раз (по умолчанию `minOccurs="1" maxOccurs="1"`)
`minOccurs="0"`	необязательный: 0 или 1 раз
`maxOccurs="unbounded"`	сколько угодно раз (не меньше `minOccurs`)
`minOccurs="0" maxOccurs="unbounded"`	от нуля до бесконечности

Атрибуты

Параметр `xs:attribute`	Смысл
`name`, `type`	имя и простой тип атрибута
`use="optional"`	необязательный (так по умолчанию)
`use="required"`	обязательный
`default="false"`	если атрибута нет, считается, что он равен этому значению
`fixed="..."`	если атрибут есть, он обязан быть равен этому значению

Именованный и анонимный сложный тип

Как и простые, сложные типы бывают именованными (`Item` — можно использовать в разных местах) и анонимными (тип элемента `order` и тип элемента `items` описаны прямо внутри них).

ok-order.xml ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order">
    <number>ORD-000042</number>
    <items>
        <item sku="BOK-0001">
            <name>Книга «Изучаем XSD»</name>
            <quantity>1</quantity>
            <price>890.00</price>
        </item>
        <item sku="MUG-0007" gift="true">
            <name>Кружка</name>
            <quantity>2</quantity>
            <price>350.50</price>
        </item>
    </items>
    <comment>Позвонить за час</comment>
</order>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-missing-sku.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order">
    <number>ORD-000042</number>
    <items>
        <item>
            <name>Книга</name>
            <quantity>1</quantity>
            <price>890.00</price>
        </item>
    </items>
</order>
```

Сообщение валидатора (строка 5)

`cvc-complex-type.4: Attribute 'sku' must appear on element 'item'.`

Атрибут `sku` объявлен с `use="required"`.

bad-wrong-order.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order">
    <number>ORD-000042</number>
    <items>
        <item sku="BOK-0001">
            <quantity>1</quantity>
            <name>Книга</name>
            <price>890.00</price>
        </item>
    </items>
</order>
```

Сообщение валидатора (строка 6)

`cvc-complex-type.2.4.a: Invalid content was found starting with element '{"urn:shop:order":quantity}'. One of '{"urn:shop:order":name}' is expected.`

В `xs:sequence` сначала `name`, потом `quantity`. Порядок нарушен.

bad-no-items.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order">
    <number>ORD-000042</number>
    <items/>
</order>
```

Сообщение валидатора (строка 4)

`cvc-complex-type.2.4.b: The content of element 'items' is not complete. One of '{"urn:shop:order":item}' is expected.`

`item` объявлен с `minOccurs="1"`: хотя бы одна позиция обязательна.

bad-extra-element.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order">
    <number>ORD-000042</number>
    <items>
        <item sku="BOK-0001">
            <name>Книга</name>
            <quantity>1</quantity>
            <price>890.00</price>
            <color>красный</color>
        </item>
    </items>
</order>
```

Сообщение валидатора (строка 9)

`cvc-complex-type.2.4.d: Invalid content was found starting with element 'color'. No child element is expected at this point.`

Элемент `color` в типе `Item` не объявлен. После `price` ничего не ожидается.

Наш XsdSchemaValidator: схема проходит все три правила. Атрибут `gift` с типом `xs:boolean` правила не нарушает: `boolean` не входит в наши списки.

9 choice и all

> Цель шага: узнать две другие модели содержимого — «один из» и «все в любом порядке».

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:simpleType name="Phone">
        <xs:restriction base="xs:string">
            <xs:pattern value="\+7[0-9]{10}"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Email">
        <xs:restriction base="xs:string">
            <xs:maxLength value="254"/>
            <xs:pattern value="[^@\s]+@[^@\s]+\.[a-z]{2,}"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Size">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="5"/>
            <xs:fractionDigits value="1"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- choice: ровно ОДИН из вариантов -->
    <xs:complexType name="Contact">
        <xs:choice>
            <xs:element name="phone" type="o:Phone"/>
            <xs:element name="email" type="o:Email"/>
        </xs:choice>
    </xs:complexType>

    <!-- all: каждый не больше одного раза, в ЛЮБОМ порядке -->
    <xs:complexType name="Box">
        <xs:all>
            <xs:element name="width"  type="o:Size"/>
            <xs:element name="height" type="o:Size"/>
            <xs:element name="depth"  type="o:Size"/>
        </xs:all>
    </xs:complexType>

    <xs:element name="contact" type="o:Contact"/>
    <xs:element name="box"     type="o:Box"/>

</xs:schema>
```

Модель	Правило	Пример
`xs:sequence`	все элементы, строго по порядку	позиция: name → quantity → price
`xs:choice`	ровно **один** из перечисленных вариантов	контакт: телефон **или** почта
`xs:all`	все элементы, в **любом** порядке, каждый не больше одного раза	размеры: width, height, depth

Модели можно вкладывать друг в друга: `xs:choice` внутри `xs:sequence` и наоборот. У `xs:all` есть ограничения: внутри только элементы (не другие модели), и каждый максимум один раз.

ok-contact-email.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<contact xmlns="urn:shop:order">
    <email>ivan@example.ru</email>
</contact>
```

ok-box-any-order.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<box xmlns="urn:shop:order">
    <depth>10</depth>
    <width>30.5</width>
    <height>20</height>
</box>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-contact-both.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<contact xmlns="urn:shop:order">
    <phone>+79991234567</phone>
    <email>ivan@example.ru</email>
</contact>
```

Сообщение валидатора (строка 4)

```
cvc-complex-type.2.4.d: Invalid content was found starting with element 'email'. No child element is expected at this point.
```

`xs:choice` разрешает только один вариант: после phone уже ничего не ожидается.

bad-box-missing.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<box xmlns="urn:shop:order">
    <width>30</width>
    <height>20</height>
</box>
```

Сообщение валидатора (строка 5)

```
cvc-complex-type.2.4.b: The content of element 'box' is not complete. One of '{"urn:shop:order":depth}' is expected.
```

В `xs:all` порядок свободный, но все элементы без `minOccurs="0"` обязательны.

10 Текст + атрибуты и наследование сложных типов

> Цель шага: разобраться с `simpleContent`, `complexContent`, `extension` и понять, почему наш валидатор по-разному относится к `extension` и `restriction` от `xs:anyType`.

Задача: текст и атрибут одновременно

Хотим так: `<price currency="RUB">1500.00</price>`. Простой тип не подходит: у него не бывает атрибутов. Обычный сложный тип тоже: в нём дочерние элементы, а нам нужен текст. Для этого есть **`xs:simpleContent`** — «сложный тип, содержимое которого — простой текст».

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Currency">
        <xs:restriction base="xs:string">
            <xs:enumeration value="RUB"/>
            <xs:enumeration value="USD"/>
            <xs:enumeration value="EUR"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Text100">
        <xs:restriction base="xs:string">
            <xs:maxLength value="100"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Coordinate">
        <xs:restriction base="xs:decimal">
            <xs:minInclusive value="-180"/>
            <xs:maxInclusive value="180"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- simpleContent: текст (сумма) + атрибут (валюта) -->
    <xs:complexType name="Money">
        <xs:simpleContent>
            <xs:extension base="o:Amount">
                <xs:attribute name="currency" type="o:Currency" use="required"/>
            </xs:extension>
        </xs:simpleContent>
    </xs:complexType>

    <!-- Базовый адрес -->
    <xs:complexType name="Address">
        <xs:sequence>
            <xs:element name="city"   type="o:Text100"/>
            <xs:element name="street" type="o:Text100"/>
        </xs:sequence>
    </xs:complexType>

    <!-- complexContent + extension: всё из Address + ещё элемент в конце -->
    <xs:complexType name="DeliveryAddress">
        <xs:complexContent>
            <xs:extension base="o:Address">
                <xs:sequence>
                    <xs:element name="entrance" type="o:Text100" minOccurs="0"/>
                </xs:sequence>
            </xs:extension>
        </xs:complexContent>
    </xs:complexType>

    <!-- Полная (длинная) форма записи обычного complexType -->
    <xs:complexType name="GeoPoint">
        <xs:complexContent>
            <xs:restriction base="xs:anyType">
                <xs:sequence>
                    <xs:element name="lat" type="o:Coordinate"/>
                    <xs:element name="lon" type="o:Coordinate"/>
                </xs:sequence>
            </xs:restriction>
        </xs:complexContent>
```

```
    </xs:complexType>

    <xs:element name="price"    type="o:Money"/>
    <xs:element name="delivery" type="o:DeliveryAddress"/>
    <xs:element name="geo"      type="o:GeoPoint"/>

</xs:schema>
```

simpleContent + extension: тип Money

```
<xs:complexType name="Money">
    <xs:simpleContent>                ← внутри элемента будет только текст
        <xs:extension base="o:Amount">    ← текст — по типу Amount, а мы ДОБАВЛЯЕМ к нему…
            <xs:attribute name="currency" .../>  ← …атрибут
        </xs:extension>
    </xs:simpleContent>
</xs:complexType>
```

`extension` значит «расширение»: берём тип из `base` и **добавляем** к нему что-то (здесь атрибут). Сузить текст extension не может. Поэтому ограничивать нужно сам базовый тип: `base="o:Amount"`, а не `base="xs:decimal"`.

complexContent + extension: наследование

`DeliveryAddress` — это `Address` плюс необязательный элемент `entrance`. `xs:complexContent` значит «содержимое — элементы». Через `extension` новые элементы добавляются **в конец** базовых: в XML сначала `city` и `street`, потом `entrance`.

Длинная форма обычного complexType

Любой сложный тип в XML Schema от чего-то наследуется. Если базу не указали, она — `xs:anyType`, вершина дерева типов. Тип `GeoPoint` записан в длинной форме, и она **полностью равнозначна** короткой:

Длинная форма

```
<xs:complexType name="GeoPoint">
  <xs:complexContent>
    <xs:restriction base="xs:anyType">
      <xs:sequence>...</xs:sequence>
    </xs:restriction>
  </xs:complexContent>
</xs:complexType>
```

Короткая форма — то же самое

```
<xs:complexType name="GeoPoint">
  <xs:sequence>...</xs:sequence>
</xs:complexType>
```

`restriction` от `xs:anyType` значит «из всего возможного разрешаем только вот это», то есть обычный строгий тип. А `extension` от `xs:anyType` значит «всё возможное плюс ещё что-то», то есть разрешено что угодно.

Четыре комбинации

	`extension` — добавить	`restriction` — сузить
`simpleContent` (текст)	текст по базовому типу + атрибуты. **Самый частый случай**, как `Money`	сузить текст у типа, который уже с simpleContent
`complexContent` (элементы)	наследование: базовые элементы + новые в конце, как `DeliveryAddress`	сузить сложный тип; от `xs:anyType` — это просто длинная форма обычного типа

ok-price.xml — ✔ проходит проверку

```
<?xml version="1.0" encoding="UTF-8"?>
<price xmlns="urn:shop:order"
  currency="RUB">1500.00</price>
```

ok-delivery.xml — ✔ проходит проверку

```
<?xml version="1.0" encoding="UTF-8"?>
<delivery xmlns="urn:shop:order">
    <city>Москва</city>
    <street>Тверская, 1</street>
    <entrance>3</entrance>
</delivery>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-price-no-currency.xml — ✘ ошибка

```
<?xml version="1.0" encoding="UTF-8"?>
<price xmlns="urn:shop:order">1500.00</price>
```

Сообщение валидатора (строка 2)

```
cvc-complex-type.4: Attribute 'currency' must appear on element 'price'.
```

Атрибут currency обязательный.

bad-price-currency.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<price xmlns="urn:shop:order" currency="GBP">1500.00</price>
```

Сообщение валидатора (строка 2)

```
cvc-enumeration-valid: Value 'GBP' is not facet-valid with respect to enumeration '[RUB, USD, EUR]'. It must be a value from the enumeration.
```

GBP нет в перечислении `Currency`.

bad-delivery-order.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<delivery xmlns="urn:shop:order">
    <entrance>3</entrance>
    <city>Москва</city>
    <street>Тверская, 1</street>
</delivery>
```

Сообщение валидатора (строка 3)

```
cvc-complex-type.2.4.a: Invalid content was found starting with element '{"urn:shop:order":entrance}'. One of '{"urn:shop:order":city}' is expected.
```

При `extension` новые элементы идут **после** базовых.

Наш XsdSchemaValidator: схема проходит все три правила. В том числе `restriction base="xs:anyType"`: это длинная форма обычного типа, наш валидатор её разрешает.

КАК НА ЭТО СМОТРИТ НАШ ВАЛИДАТОР

- `<xs:extension base="xs:string">` или `base="xs:decimal"` внутри `simpleContent` — нарушение правила 1 или 2: текст остаётся неограниченным.
- `<xs:extension base="xs:anyType">` — нарушение правила 3: разрешено что угодно.
- `<xs:restriction base="xs:anyType">` внутри `complexContent` — **не нарушение**.

11 Повторное использование

> Цель шага: узнать `ref`, `xs:group`, `xs:attributeGroup` и `substitutionGroup`, а в конце собрать полную картину: откуда объявление получает тип.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:simpleType name="Text100">
        <xs:restriction base="xs:string">
            <xs:maxLength value="100"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Phone">
        <xs:restriction base="xs:string">
            <xs:pattern value="\+7[0-9]{10}"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="CardMask">
        <xs:restriction base="xs:string">
            <xs:pattern value="[0-9]{4}\*{8}[0-9]{4}"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- group: именованный кусок содержимого -->
    <xs:group name="ContactGroup">
        <xs:sequence>
            <xs:element name="phone" type="o:Phone"/>
            <xs:element name="email" type="o:Text100" minOccurs="0"/>
        </xs:sequence>
    </xs:group>

    <!-- attributeGroup: именованный набор атрибутов -->
    <xs:attributeGroup name="Audit">
        <xs:attribute name="createdBy" type="o:Text100" use="required"/>
        <xs:attribute name="version"   type="xs:int"/>
    </xs:attributeGroup>

    <!-- Глобальный элемент: его можно подключать через ref -->
    <xs:element name="note" type="o:Text100"/>

    <!-- substitutionGroup: голова группы и её «заместители» -->
    <xs:complexType name="Payment">
        <xs:sequence>
            <xs:element name="amount" type="o:Amount"/>
        </xs:sequence>
    </xs:complexType>
    <xs:complexType name="CardPayment">
        <xs:complexContent>
            <xs:extension base="o:Payment">
                <xs:sequence>
                    <xs:element name="cardMask" type="o:CardMask"/>
                </xs:sequence>
            </xs:extension>
        </xs:complexContent>
    </xs:complexType>

    <xs:element name="payment"     type="o:Payment" abstract="true"/>
    <xs:element name="cardPayment" type="o:CardPayment" substitutionGroup="o:payment"/>
    <xs:element name="cashPayment" substitutionGroup="o:payment"/>   <!-- тип берётся у головы -->

    <xs:element name="order">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="customer">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="name" type="o:Text100"/>
                            <xs:group ref="o:ContactGroup"/>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
```

```
            <xs:element ref="o:payment"/>
            <xs:element ref="o:note" minOccurs="0"/>
        </xs:sequence>
        <xs:attributeGroup ref="o:Audit"/>
    </xs:complexType>
</xs:element>

</xs:schema>
```

`ref`: подключить готовый глобальный элемент

`<xs:element ref="o:note"/>` вставляет в это место глобальный элемент `note` целиком, с его именем и типом. В XML будет `<note>`.

	`type="o:X"`	`ref="o:x"`
Что берём	только **тип**, имя элемента задаём своё	готовый **элемент**: и имя, и тип
На что ссылается	на `xs:simpleType` / `xs:complexType`	на глобальный `xs:element`
Можно вместе с `name`	**да, обязательно**	**нет**

`xs:group` и `xs:attributeGroup`

- `xs:group` — именованный кусок модели содержимого. `<xs:group ref="o:ContactGroup"/>` вставляет в это место `phone` и `email`.
- `xs:attributeGroup` — именованный набор атрибутов. `<xs:attributeGroup ref="o:Audit"/>` добавляет элементу `createdBy` и `version`.

Это «копипаста по ссылке»: в XML никаких следов группы нет, там просто появляются её элементы и атрибуты.

`substitutionGroup`: взаимозаменяемые элементы

Оплата бывает картой или наличными. Делаем элемент-«голову» `payment` и элементы-заместители, которые могут стоять **на его месте**:

- `abstract="true"` у головы — сам `<payment>` писать нельзя, только заместителей.
- `cardPayment` — свой тип `CardPayment`, наследник `Payment`. Тип заместителя обязан быть производным от типа головы.
- `cashPayment` — **без** `type`. Тогда он получает тип головы, `Payment`.

В модели содержимого стоит `<xs:element ref="o:payment"/>`, а в XML вместо него пишут `<cardPayment>` или `<cashPayment>`.

ok-card.xml — ✔ проходит проверку

```
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order" createdBy="site" version="2">
    <customer>
        <name>Иван Петров</name>
        <phone>+79991234567</phone>
        <email>ivan@example.ru</email>
    </customer>
    <cardPayment>
        <amount>1500.00</amount>
        <cardMask>1234********5678</cardMask>
    </cardPayment>
    <note>Позвонить заранее</note>
</order>
```

ok-cash.xml — ✔ проходит проверку

```
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order" createdBy="operator">
    <customer>
        <name>Анна</name>
        <phone>+79990000000</phone>
    </customer>
    <cashPayment>
        <amount>700.00</amount>
    </cashPayment>
</order>
```

ПОПРОБУЙТЕ СЛОМАТЬ

bad-abstract-payment.xml — ✘ ошибка

```
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order" createdBy="site">
    <customer>
        <name>Анна</name>
        <phone>+79990000000</phone>
    </customer>
    <payment>
        <amount>700.00</amount>
    </payment>
</order>
```

Сообщение валидатора (строка 7)

```
cvc-elt.2: The value of {abstract} in the element declaration for 'payment' must be false.
```

Голова группы объявлена `abstract="true"`: сам <payment> использовать нельзя.

bad-missing-createdby.xml ✘ ошибка

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order">
    <customer>
        <name>Анна</name>
        <phone>+79990000000</phone>
    </customer>
    <cashPayment>
        <amount>700.00</amount>
    </cashPayment>
</order>
```

Сообщение валидатора (строка 2)

```
cvc-complex-type.4: Attribute 'createdBy' must appear on element 'order'.
```

`createdBy` пришёл из `attributeGroup` и там объявлен обязательным.

Наш XsdSchemaValidator: схема проходит все три правила. `cashPayment` без `type` не нарушает правило 3: тип ему дала голова группы.

Итог: откуда объявление получает тип

Теперь известны все кусочки, и можно ответить на вопрос, который был неочевиден в начале. У каждого `xs:element` и `xs:attribute` есть тип, и получить его объявление может одним из способов:

#	Как записано	Откуда тип	Где это проверяет наш валидатор
1	`type="xs:string"`	встроенный тип по имени	здесь же: `checkTypeRef`
2	`type="o:Name"`	свой именованный тип	в объявлении `Name`
3	внутри `<xs:simpleType>` или `<xs:complexType>`	анонимный тип на месте	во вложенных тегах: `restriction`, `extension`, `element`…
4	ничего из перечисленного	неявный: `xs:anyType` у элемента, `xs:anySimpleType` у атрибута	здесь же: нарушение правила 3
+	`ref="o:x"`	из глобального объявления `x`	в объявлении `x`
+	без `type`, но с `substitutionGroup`	от головы группы	в объявлении головы

Именно в таком порядке их разбирает метод `checkElementOrAttribute`: сначала `ref`, потом `type`, потом вложенный тип, и только если нет ничего — неявный `anyType` (с исключением для `substitutionGroup`).

12 «Всеядные» конструкции

> Цель шага: понять, что такое `xs:anyType`, `xs:anySimpleType` и `xs:any` и почему правило 3 их запрещает.

Три конструкции говорят валидатору «здесь не проверяй»:

Конструкция	Что разрешает	Как появляется
`xs:anyType`	что угодно: любой текст, любые вложенные элементы и атрибуты	явно `type="xs:anyType"`; **неявно** у элемента без типа; через `extension base="xs:anyType"`
`xs:anySimpleType`	любой текст	явно `type="xs:anySimpleType"`; **неявно** у атрибута без типа
`<xs:any/>`	в этом месте модели может стоять любой элемент	только явно

У `xs:any` есть параметр `processContents`: `strict` — элемент обязан быть где-то объявлен, `lax` — проверить, если объявление найдётся, `skip` — не проверять вообще. Для атрибутов есть аналог — `<xs:anyAttribute/>`.

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:element name="request">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="raw" type="xs:anyType"/>          <!-- явно anyType -->
                <xs:element name="untyped"/>                         <!-- неявно anyType -->
                <xs:element name="extra">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:any processContents="lax" minOccurs="0" maxOccurs="unbounded"/>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
            </xs:sequence>
            <xs:attribute name="flag"/>                              <!-- неявно anySimpleType -->
        </xs:complexType>
    </xs:element>

</xs:schema>
```

Эта схема абсолютно корректна, этап 1 её пропускает. И вот что проходит проверку:

ok-anything-goes.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<request xmlns="urn:shop:order" flag="что угодно">
    <raw><whatever><deep>любая вложенность</deep></whatever></raw>
    <untyped>тоже что угодно <b>с разметкой</b></untyped>
    <extra>
        <password>qwerty</password>
        <script>alert(1)</script>
    </extra>
</request>
```

ПОЧЕМУ ЭТО ПЛОХО

Пароль, скрипт, разметка внутри текста — валидатор всё принял. Схема с такими конструкциями ничего не гарантирует: потребитель данных не знает, что придёт в этих местах. Поэтому **правило 3** запрещает их везде, включая неявные случаи. Неявные опаснее всего: забыли написать `type` — и элемент молча стал «принимать всё».

Наш XsdSchemaValidator отклонит эту схему:

```
[schema.xsd]: <xs:element name="raw"> type="xs:anyType": использование xs:anyType запрещено
```

Обратите внимание: валидатор останавливается на первом нарушении. Если его исправить, следующим он сообщит про `untyped`, потом про `xs:any` и про атрибут `flag`.

13 Схема из нескольких файлов

> Цель шага: разложить схему по файлам с помощью `xs:include` и `xs:import` и собрать итоговую схему заказа из всего, что мы изучили.

include и import

	`xs:include`	`xs:import`
Когда	подключаемый файл — **часть той же** схемы, тот же `targetNamespace`	подключаемый файл — **другая** схема, другое пространство имён
Запись	`<xs:include schemaLocation="address.xsd"/>`	`<xs:import namespace="urn:shop:common" schemaLocation="common.xsd"/>`
Как ссылаться на типы	как на свои: `o:Address`	через префикс того пространства: нужен `xmlns:c="urn:shop:common"`, затем `c:Money`
Аналогия	разрезать один большой файл на куски	`import` чужого пакета в Java

`schemaLocation` — где лежит файл. Найти его по этому пути — задача **resolver**'а: в наших классах это `FilePathResolver` (абсолютные пути; относительные — от папки главного файла) и `ClassPathResolver` (пути от корня classpath). В примере пути относительные, файлы лежат рядом.

```
order.xsd                 targetNamespace = urn:shop:order
├── xs:include → address.xsd    targetNamespace = urn:shop:order   (Address, DeliveryAddress, Zip, Text100)
└── xs:import  → common.xsd     targetNamespace = urn:shop:common  (Amount, Currency, Money)
```

common.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Общие типы магазина. Своё пространство имён: urn:shop:common -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:c="urn:shop:common"
           targetNamespace="urn:shop:common"
           elementFormDefault="qualified">

    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="Currency">
        <xs:restriction base="xs:string">
            <xs:enumeration value="RUB"/>
            <xs:enumeration value="USD"/>
            <xs:enumeration value="EUR"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:complexType name="Money">
        <xs:simpleContent>
            <xs:extension base="c:Amount">
                <xs:attribute name="currency" type="c:Currency" use="required"/>
            </xs:extension>
        </xs:simpleContent>
    </xs:complexType>

</xs:schema>
```

address.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Часть схемы заказа: то же пространство имён, что у order.xsd -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <xs:simpleType name="Text100">
        <xs:restriction base="xs:string">
            <xs:maxLength value="100"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="Zip">
        <xs:restriction base="xs:string">
            <xs:length value="6"/>
            <xs:pattern value="[0-9]+"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:complexType name="Address">
        <xs:sequence>
            <xs:element name="zip"    type="o:Zip"/>
            <xs:element name="city"   type="o:Text100"/>
            <xs:element name="street" type="o:Text100"/>
        </xs:sequence>
    </xs:complexType>

    <xs:complexType name="DeliveryAddress">
        <xs:complexContent>
            <xs:extension base="o:Address">
                <xs:sequence>
                    <xs:element name="entrance" type="o:Text100" minOccurs="0"/>
                </xs:sequence>
            </xs:extension>
        </xs:complexContent>
    </xs:complexType>

</xs:schema>
```

order.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Главная схема заказа -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:o="urn:shop:order"
           xmlns:c="urn:shop:common"
           targetNamespace="urn:shop:order"
           elementFormDefault="qualified">

    <!-- Другое пространство имён — import -->
    <xs:import namespace="urn:shop:common" schemaLocation="common.xsd"/>
    <!-- То же пространство имён — include -->
    <xs:include schemaLocation="address.xsd"/>

    <!-- ===== Простые типы ===== -->
    <xs:simpleType name="OrderNumber">
        <xs:restriction base="xs:string">
            <xs:pattern value="ORD-[0-9]{6}"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="Status">
        <xs:restriction base="xs:string">
            <xs:enumeration value="NEW"/>
            <xs:enumeration value="PAID"/>
            <xs:enumeration value="SHIPPED"/>
            <xs:enumeration value="CANCELLED"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="ShortName">
        <xs:restriction base="o:Text100">          <!-- Text100 — из address.xsd -->
            <xs:maxLength value="30"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="Phone">
        <xs:restriction base="xs:string">
            <xs:pattern value="\+7[0-9]{10}"/>
        </xs:restriction>
    </xs:simpleType>
```

```xml

    <xs:simpleType name="Sku">
        <xs:restriction base="xs:string">
            <xs:pattern value="[A-Z]{3}-[0-9]{4}"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="Quantity">
        <xs:restriction base="xs:positiveInteger">
            <xs:minInclusive value="1"/>
            <xs:maxInclusive value="999"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="TagCode">
        <xs:restriction base="xs:string">
            <xs:pattern value="[a-z]{2,10}"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="Tags">
        <xs:list itemType="o:TagCode"/>
    </xs:simpleType>

    <xs:simpleType name="Asap">
        <xs:restriction base="xs:string">
            <xs:enumeration value="ASAP"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="DeliveryDate">
        <xs:union memberTypes="xs:date o:Asap"/>
    </xs:simpleType>

    <xs:simpleType name="CardMask">
        <xs:restriction base="xs:string">
            <xs:pattern value="[0-9]{4}\*{8}[0-9]{4}"/>
        </xs:restriction>
    </xs:simpleType>

    <!-- ===== Переиспользуемые куски ===== -->
    <xs:group name="ContactGroup">
        <xs:sequence>
            <xs:element name="phone" type="o:Phone"/>
            <xs:element name="email" type="o:Text100" minOccurs="0"/>
        </xs:sequence>
    </xs:group>

    <xs:attributeGroup name="Audit">
        <xs:attribute name="createdBy" type="o:ShortName" use="required"/>
        <xs:attribute name="version"   type="xs:int"/>
    </xs:attributeGroup>

    <!-- ===== Сложные типы ===== -->
    <xs:complexType name="Item">
        <xs:sequence>
            <xs:element name="name"     type="o:Text100"/>
            <xs:element name="quantity" type="o:Quantity"/>
            <xs:element name="price"    type="c:Money"/>     <!-- тип из common.xsd -->
        </xs:sequence>
        <xs:attribute name="sku"  type="o:Sku" use="required"/>
        <xs:attribute name="gift" type="xs:boolean" default="false"/>
    </xs:complexType>

    <xs:complexType name="Payment">
        <xs:sequence>
            <xs:element name="amount" type="c:Money"/>
        </xs:sequence>
    </xs:complexType>

    <xs:complexType name="CardPayment">
        <xs:complexContent>
            <xs:extension base="o:Payment">
                <xs:sequence>
                    <xs:element name="cardMask" type="o:CardMask"/>
                </xs:sequence>
            </xs:extension>
        </xs:complexContent>
    </xs:complexType>

    <!-- ===== Глобальные элементы ===== -->
    <xs:element name="payment"     type="o:Payment" abstract="true"/>
    <xs:element name="cardPayment" type="o:CardPayment" substitutionGroup="o:payment"/>
    <xs:element name="cashPayment" substitutionGroup="o:payment"/>

```

```xml
    <xs:element name="order">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="number" type="o:OrderNumber"/>
                <xs:element name="status" type="o:Status"/>
                <xs:element name="customer">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="name" type="o:Text100"/>
                            <xs:group ref="o:ContactGroup"/>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
                <xs:element name="delivery"     type="o:DeliveryAddress"/>
                <xs:element name="deliveryDate" type="o:DeliveryDate"/>
                <xs:element name="items">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="item" type="o:Item" maxOccurs="unbounded"/>
                        </xs:sequence>
                    </xs:complexType>
                </xs:element>
                <xs:element ref="o:payment"/>
                <xs:element name="tags" type="o:Tags" minOccurs="0"/>
                <xs:element name="comment" minOccurs="0">
                    <xs:simpleType>
                        <xs:restriction base="xs:string">
                            <xs:maxLength value="500"/>
                        </xs:restriction>
                    </xs:simpleType>
                </xs:element>
            </xs:sequence>
            <xs:attributeGroup ref="o:Audit"/>
        </xs:complexType>
    </xs:element>

</xs:schema>
```

Итоговый заказ

ok-order.xml — ✔ проходит проверку

```xml
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:shop:order"
       xmlns:c="urn:shop:common"
       createdBy="site" version="1">
    <number>ORD-000042</number>
    <status>PAID</status>
    <customer>
        <name>Иван Петров</name>
        <phone>+79991234567</phone>
        <email>ivan@example.ru</email>
    </customer>
    <delivery>
        <zip>101000</zip>
        <city>Москва</city>
        <street>Тверская, 1</street>
        <entrance>3</entrance>
    </delivery>
    <deliveryDate>ASAP</deliveryDate>
    <items>
        <item sku="BOK-0001">
            <name>Книга «Изучаем XSD»</name>
            <quantity>1</quantity>
            <price currency="RUB">890.00</price>
        </item>
        <item sku="MUG-0007" gift="true">
            <name>Кружка</name>
            <quantity>2</quantity>
            <price currency="RUB">350.50</price>
        </item>
    </items>
    <cardPayment>
        <amount currency="RUB">1591.00</amount>
        <cardMask>1234********5678</cardMask>
    </cardPayment>
    <tags>gift urgent</tags>
    <comment>Позвонить за час</comment>
</order>
```

Заметьте: в XML нет ни одного префикса `c:`, хотя `Money` — тип из другого пространства имён. Пространство имён определяет, в какое пространство попадает **элемент**, а элементы `price` и `amount` объявлены в `order.xsd`, то есть в `urn:shop:order`. Из `common.xsd` взят только их **тип**.

ПОПРОБУЙТЕ СЛОМАТЬ

Ниже показана только суть изменения, файлы целиком — копии итогового заказа.

Что изменили	Сообщение валидатора
`<status>DONE</status>`	`cvc-enumeration-valid: Value 'DONE' is not facet-valid with respect to enumeration '[NEW, PAID, SHIPPED, CANCELLED]'. It must be a value from the enumeration.`
`<price>890.00</price>` (без валюты)	`cvc-complex-type.4: Attribute 'currency' must appear on element 'price'.` Правило пришло из `common.xsd`.
`<zip>10100</zip>`	`cvc-length-valid: Value '10100' with length = '5' is not facet-valid with respect to length '6' for type 'Zip'.` Правило из `address.xsd`.
удалили `<cardPayment>`	`cvc-complex-type.2.4.a: Invalid content was found starting with element '{"urn:shop:order":tags}'. One of '{"urn:shop:order":payment}' is expected.` Ожидается `payment`, то есть любой из его заместителей.

Наш XsdSchemaValidator: схема проходит все три правила. Проверены все три файла: `XsdSchema.files()` содержит `order.xsd`, `address.xsd` и `common.xsd`.

ЧАСТЬ IV

Наш валидатор

Теперь, когда понятна каждая конструкция, разберём, что именно делает наш код: что ловит этап 1, какие три правила проверяет этап 2 и как код обходит схему.

Два этапа проверки схемы

```
order.xsd + подключаемые файлы
     │
     ▼
XsdSchemaCreator   ── этап 1: SchemaFactory.newSchema() компилирует схему.
     │                  Resolver подгружает xs:include / xs:import и складывает файлы в мапу.
     ▼
XsdSchema(schema, files)
     │
     ├──► XsdSchemaValidator.validate(xsdSchema)   ── этап 2: наши три правила по каждому файлу из files
     │
     └──► schema.newValidator().validate(xml)       ── проверка XML-документов по схеме
```

Не путайте: **этап 1 и этап 2 проверяют схему**, а `Validator` из последней строки проверяет **XML-документы** по уже проверенной схеме. Все красные блоки в шагах 1–13 — это ошибки `Validator`.

Этап 1: схема корректна по стандарту

Компилятор XML Schema (встроенный в JDK Xerces) проверяет, что схема вообще имеет смысл. Если нет — `XsdSchemaCreator` бросает исключение, и до этапа 2 дело не доходит. Что он ловит:

Ошибка в схеме	Сообщение компилятора
опечатка в имени тега: `<xs:elemnt>`	`s4s-elt-invalid-content.1: The content of 'schema' is invalid. Element 'elemnt' is invalid, misplaced, or occurs too often.`
несуществующий атрибут: `size="10"` у `xs:element`	`s4s-att-not-allowed: Attribute 'size' cannot appear in element 'element'.`
ссылка на необъявленный тип	`src-resolve: Cannot resolve the name 'o:NoSuchType' to a(n) 'type definition' component.`
свой тип без префикса: `type="Name"` вместо `o:Name`	`src-resolve.4.1: Error resolving component 'Name'. It was detected that 'Name' has no namespace, but components with no target namespace are not referenceable from schema document 'null'. If 'Name' is intended to have a namespace, perhaps a prefix needs to be provided. If it is intended that 'Name' has no namespace, then an 'import' without a "namespace" attribute should be added to 'null'.`
фасет не подходит к типу: `maxLength` у `xs:decimal`	`cos-applicable-facets: Facet 'maxLength' is not allowed by type Amount.`
попытка расширить при наследовании: `maxLength="200"` поверх `100`	`maxLength-valid-restriction: In the definition of LongName, maxLength value = '200' must be <= that of the base type '100'.`
и `type`, и вложенный тип у одного элемента	`src-element.3: Element 'note' has both a 'type' attribute and a 'anonymous type' child. Only one of these is allowed for an element.`
подключаемый файл не найден	`schema_reference: Failed to read schema document 'no-such-file.xsd', because 'file' access is not allowed due to restriction set by the accessExternalSchema property.` Resolver не нашёл файл и вернул `null`, после чего парсер попробовал загрузить его сам, но `ACCESS_EXTERNAL_SCHEMA = ""` это запрещает.

Этап 2: схема не разрешает слишком много

Схема может быть идеально корректной и при этом пропускать что угодно (вспомните шаг 12). Этап 2 проверяет три правила, которые стандарт XML Schema не требует, а требует наш проект:

№	Правило	Почему
1	Строковый тип ограничен `length`, `maxLength`, `enumeration` или `pattern`	`xs:string` без ограничений — строка любой длины (шаги 4–5)
2	Тип на основе `decimal`, `integer`, `positiveInteger`, `nonNegativeInteger`, `negativeInteger`, `nonPositiveInteger` имеет `totalDigits` или пару min + max	у этих типов нет верхнего предела (шаги 4, 6)
3	Нет `xs:any`, `xs:anyType`, `xs:anySimpleType`, в том числе неявных	в этих местах проверка отключена (шаг 12)

Три правила на примерах

Слева — нарушение, справа — исправление, внизу — сообщение нашего `XsdSchemaValidator`. Каждая пара проверена: левая схема отклоняется, правая проходит.

Правило 1: строки

Встроенная строка указана напрямую — к ней некуда добавить ограничения

Нарушение ✘

```xml
<xs:element name="name" type="xs:string"/>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:element name="name" type="o:Name"/>
```

```
[r1-type-string-bad.xsd]: <xs:element name="name"> type="xs:string": для строкового типа должно быть задано ограничение (xs:length, xs:maxLength, xs:enumeration или xs:pattern) через xs:restriction
```

Есть ограничение, но только снизу

Нарушение ✘

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:minLength value="1"/>
    </xs:restriction>
</xs:simpleType>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:minLength value="1"/>
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>
```

```
[r1-minlength-only-bad.xsd]: <xs:restriction base="xs:string"> (в <xs:simpleType name="Name">): не задан ни один из параметров xs:length, xs:maxLength, xs:enumeration, xs:pattern
```

simpleContent: extension не ограничивает текст

Нарушение ✘

```xml
<xs:complexType name="Note">
    <xs:simpleContent>
        <xs:extension base="xs:string">
            <xs:attribute name="lang"
  type="xs:language"/>
        </xs:extension>
    </xs:simpleContent>
</xs:complexType>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:complexType name="Note">
    <xs:simpleContent>
        <xs:extension base="o:Name">
            <xs:attribute name="lang"
  type="xs:language"/>
        </xs:extension>
    </xs:simpleContent>
</xs:complexType>
```

```
[r1-extension-string-bad.xsd]: <xs:extension> base="xs:string" (в <xs:complexType name="Note">): для строкового типа должно быть задано ограничение (xs:length, xs:maxLength, xs:enumeration или xs:pattern) через xs:restriction
```

Список из неограниченных строк

Нарушение ✘

```xml
<xs:simpleType name="Tags">
    <xs:list itemType="xs:string"/>
</xs:simpleType>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:simpleType name="Tags">
    <xs:list itemType="o:Name"/>
</xs:simpleType>
```

```
[r1-list-string-bad.xsd]: <xs:list> itemType="xs:string" (в <xs:simpleType name="Tags">): для строкового типа должно быть задано ограничение (xs:length, xs:maxLength, xs:enumeration или xs:pattern) через xs:restriction
```

Объединение, где одна ветвь — неограниченная строка

Нарушение ✘

```xml
    <xs:simpleType name="DateOrText">
        <xs:union memberTypes="xs:date xs:string"/>
    </xs:simpleType>
```

Исправление ✔

```xml
    <xs:simpleType name="Name">
        <xs:restriction base="xs:string">
            <xs:maxLength value="100"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:simpleType name="DateOrText">
        <xs:union memberTypes="xs:date o:Name"/>
    </xs:simpleType>
```

```
[r1-union-string-bad.xsd]: <xs:union> memberTypes="xs:string" (в <xs:simpleType name="DateOrText">): для строкового типа должно быть задано ограничение (xs:length, xs:maxLength, xs:enumeration или xs:pattern) через xs:restriction
```

Правило 2: числа

Встроенное число указано напрямую

Нарушение ✘

```xml
    <xs:element name="total" type="xs:decimal"/>
```

Исправление ✔

```xml
    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>

    <xs:element name="total" type="o:Amount"/>
```

```
[r2-type-decimal-bad.xsd]: <xs:element name="total"> type="xs:decimal": для числового типа должно быть задано ограничение (xs:totalDigits или xs:min* + xs:max*) через xs:restriction
```

fractionDigits ограничивает только дробную часть

Нарушение ✘

```xml
    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>
```

Исправление ✔

```xml
    <xs:simpleType name="Amount">
        <xs:restriction base="xs:decimal">
            <xs:totalDigits value="18"/>
            <xs:fractionDigits value="2"/>
        </xs:restriction>
    </xs:simpleType>
```

```
[r2-fraction-only-bad.xsd]: <xs:restriction base="xs:decimal"> (в <xs:simpleType name="Amount">): должен быть задан xs:totalDigits либо пара xs:minInclusive/xs:minExclusive и xs:maxInclusive/xs:maxExclusive
```

Есть только нижняя граница

Нарушение ✘

```xml
    <xs:simpleType name="Quantity">
        <xs:restriction base="xs:integer">
            <xs:minInclusive value="1"/>
        </xs:restriction>
    </xs:simpleType>
```

Исправление ✔

```xml
    <xs:simpleType name="Quantity">
        <xs:restriction base="xs:integer">
            <xs:minInclusive value="1"/>
            <xs:maxInclusive value="999"/>
        </xs:restriction>
    </xs:simpleType>
```

```
[r2-min-only-bad.xsd]: <xs:restriction base="xs:integer"> (в <xs:simpleType name="Quantity">): должен быть задан xs:totalDigits либо пара xs:minInclusive/xs:minExclusive и xs:maxInclusive/xs:maxExclusive
```

Правило 3: «всеядные» конструкции

xs:any — любой элемент

Нарушение ✘

```xml
<xs:element name="extra">
    <xs:complexType>
        <xs:sequence>
            <xs:any processContents="lax"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:element name="extra">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="note" type="o:Name"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

```
[r3-any-bad.xsd]: использование xs:any запрещено (в <xs:element name="extra">)
```

Явный xs:anyType

Нарушение ✘

```xml
<xs:element name="raw" type="xs:anyType"/>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:element name="raw" type="o:Name"/>
```

```
[r3-anytype-bad.xsd]: <xs:element name="raw"> type="xs:anyType": использование xs:anyType запрещено
```

Элемент без типа — это неявный xs:anyType

Нарушение ✘

```xml
<xs:element name="untyped"/>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:element name="untyped" type="o:Name"/>
```

```
[r3-implicit-element-bad.xsd]: элемент 'untyped' объявлен без типа и неявно имеет тип xs:anyType
```

Атрибут без типа — это неявный xs:anySimpleType

Нарушение ✘

```xml
<xs:element name="order">
    <xs:complexType>
        <xs:attribute name="flag"/>
    </xs:complexType>
</xs:element>
```

Исправление ✔

```xml
<xs:element name="order">
    <xs:complexType>
        <xs:attribute name="flag" type="xs:boolean"/>
    </xs:complexType>
</xs:element>
```

```
[r3-implicit-attribute-bad.xsd]: атрибут 'flag' объявлен без типа и неявно имеет тип xs:anySimpleType
```

extension от xs:anyType — разрешено всё и ещё немного. restriction — можно

Нарушение ✘

```xml
<xs:complexType name="Payload">
    <xs:complexContent>
        <xs:extension base="xs:anyType"/>
    </xs:complexContent>
</xs:complexType>
```

Исправление ✔

```xml
<xs:simpleType name="Name">
    <xs:restriction base="xs:string">
        <xs:maxLength value="100"/>
    </xs:restriction>
</xs:simpleType>

<xs:complexType name="Payload">
    <xs:complexContent>
        <xs:restriction base="xs:anyType">
            <xs:sequence>
                <xs:element name="note" type="o:Name"/>
            </xs:sequence>
        </xs:restriction>
    </xs:complexContent>
</xs:complexType>
```

```
[r3-extension-anytype-bad.xsd]: <xs:extension> base="xs:anyType" (в <xs:complexType name="Payload">): использование xs:anyType запрещено
```

Как код обходит схему

Метод `validateFile` читает файл в DOM (как обычный XML — шаг 3) и проходит по **всем** тегам из пространства XML Schema сверху вниз, включая вложенные. Для каждого тега решение принимается по его имени:

Тег	Метод	Что проверяется
`xs:element`, `xs:attribute`	`checkElementOrAttribute`	откуда тип (таблица из шага 11): `ref` → пропуск; `type` → `checkTypeRef`; вложенный тип → пропуск, его теги проверятся сами; ничего → неявный `anyType` / `anySimpleType`, кроме элемента с `substitutionGroup`
`xs:restriction`	`checkRestriction`	если `base` — встроенный тип из правил, внутри должны быть нужные фасеты. `base` — свой тип → пропуск (он проверен там, где объявлен). `base="xs:anyType"` внутри `complexContent` → можно (шаг 10)
`xs:extension`	`checkTypeRef` для `base`	extension не сужает, поэтому встроенная строка, число или `anyType` в `base` — всегда нарушение
`xs:list`	`checkTypeRef` для `itemType`	тип элемента списка
`xs:union`	`checkTypeRef` для каждого имени из `memberTypes`	каждая ветвь объединения
`xs:any`	сразу нарушение	правило 3
остальные (`sequence`, `complexType`, `group` …)	—	сами по себе ничего не ограничивают; их содержимое проверяется, когда обход дойдёт до вложенных тегов

`checkTypeRef`: ссылка на тип по имени

Вызывается для атрибутов `type`, `base` (у extension), `itemType`, `memberTypes`. Сначала `builtInTypeName` разрешает префикс (шаг 2) и выясняет, встроенный ли это тип. Если тип свой — выход: он проверяется в собственном объявлении. Если встроенный и входит в списки правил — нарушение: в ссылке по имени положить фасеты некуда, их задают только внутри `xs:restriction`.

Каждая конструкция проверяется один раз

Ошибка всегда указывает на то место, где проблема объявлена, а не на места, где тип используется. Если `o:Name` — неограниченная строка, ругаться будут на `<xs:restriction base="xs:string">` внутри `Name`, а не на каждый `type="o:Name"`.

Что валидатор сознательно не делает

- Ограничения должны быть **явными**: `xs:positiveInteger` только с `maxInclusive` — нарушение, хотя нижняя граница 1 у типа есть по стандарту.
- Проверяются только типы, названные в правилах. `xs:token`, `xs:normalizedString` (тоже строки, см. дерево в шаге 4) не проверяются.
- `xs:anyAttribute` не проверяется: его нет в требованиях.
- Останавливается на **первом** нарушении: файлы в порядке загрузки, теги сверху вниз.

Памятка

Всё самое нужное на нескольких страницах. Номер шага в скобках — где тема разобрана подробно.

XML (шаги 1–2)

Запись	Что это
`<a>текст</a>`	элемент с текстом
`<a/>`	пустой элемент
`<a id="1">`	атрибут: строка, в кавычках, не повторяется
`<!-- ... -->`	комментарий
`< & > " '`	`< & > " '`

Запись	Что это
`xmlns:o="urn:x"`	префикс `o` = пространство `urn:x`
`xmlns="urn:x"`	пространство по умолчанию для элементов без префикса
`o:order`	QName: элемент `order` из `urn:x`; префикс любой
атрибут без префикса	не в пространстве имён, принадлежит элементу

Правильно оформленный документ: один корень · все теги закрыты · вложенность не перекрывается · атрибуты в кавычках · регистр важен · `<` и `&` только через сущности · префиксы объявлены.

Скелет схемы (шаг 3)

schema.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"   <!-- язык схем -->
           xmlns:o="urn:shop:order"                      <!-- префикс для своих типов -->
           targetNamespace="urn:shop:order"              <!-- куда попадут объявления -->
           elementFormDefault="qualified">               <!-- вложенные элементы тоже в нём -->
    <!-- xs:import / xs:include — первыми -->
    <!-- затем в любом порядке: simpleType, complexType, element, attribute, group, attributeGroup -->
</xs:schema>
```

Встроенные типы (шаг 4)

```
anyType → anySimpleType ┬ string → normalizedString → token → …
                        ├ decimal → integer ┬ nonNegativeInteger → positiveInteger
                        │                   ├ nonPositiveInteger → negativeInteger
                        │                   └ long → int → short → byte
                        └ boolean · date · dateTime · time · …
синие — без верхнего предела: их требуют ограничить правила 1–2 · зелёные — уже ограничены · красные — запрещены правилом 3
```

Простые типы (шаги 5–7)

restriction — сузить

```xml
<xs:simpleType name="Code">
  <xs:restriction base="xs:string">
    <xs:maxLength value="10"/>
  </xs:restriction>
</xs:simpleType>
```

list / union / наследование

```xml
<xs:list itemType="o:Code"/>        <!-- "a b c" -->
<xs:union memberTypes="xs:date o:Asap"/>
<xs:restriction base="o:Code">  <!-- только сужать -->
<!-- анонимный: simpleType без name внутри xs:element -->
```

Фасеты (шаги 5–6)

Фасет	Для каких типов	Смысл	Правило 1/2
`length`	строки, list	точная длина (у list — число элементов)	**засчитывается (1)**
`maxLength`	строки, list	максимальная длина	**засчитывается (1)**
`minLength`	строки, list	минимальная длина	**нет**
`pattern`	все простые	регулярка на всё значение; несколько = «или»	**засчитывается (1)**
`enumeration`	все простые	значение из списка	**засчитывается (1)**
`whiteSpace`	строки	`preserve` / `replace` / `collapse`	**нет**
`totalDigits`	decimal-семейство	всего цифр	**засчитывается (2)**
`fractionDigits`	decimal-семейство	цифр после точки	**нет**
`minInclusive` / `minExclusive`	числа, даты	≥ / >	только в паре с max (2)
`maxInclusive` / `maxExclusive`	числа, даты	≤ / <	только в паре с min (2)

Параметры элементов и атрибутов (шаги 8, 11)

`xs:element`	
`name`	имя элемента
`type`	тип по имени
`ref`	взять глобальный элемент целиком
`minOccurs` / `maxOccurs`	сколько раз; по умолчанию 1/1; `unbounded`
`default` / `fixed`	значение по умолчанию / обязательное значение
`abstract="true"`	сам не используется, только заместители
`substitutionGroup`	может стоять на месте элемента-головы

`xs:attribute`	
`name` , `type`	имя; тип — всегда простой
`ref`	взять глобальный атрибут
`use`	`optional` (по умолчанию) · `required` · `prohibited`
`default`	значение, если атрибута нет
`fixed`	единственно допустимое значение

Модели содержимого (шаги 8–9)	
`xs:sequence`	все, строго по порядку
`xs:choice`	один из вариантов
`xs:all`	все, любой порядок, каждый ≤ 1 раза

Формы complexType (шаги 8, 10)

Нужно	Как записать
дочерние элементы (+ атрибуты)	`complexType` → `sequence` / `choice` / `all` → затем `attribute`
только атрибуты, без содержимого	`complexType` → только `attribute`
текст + атрибуты	`complexType` → `simpleContent` → `extension base="ограниченный простой тип"` → `attribute`
наследовать сложный тип	`complexType` → `complexContent` → `extension base="o:Base"` → новые элементы (в конец)
длинная форма обычного типа	`complexContent` → `restriction base="xs:anyType"` → модель — то же, что короткая форма

Откуда объявление получает тип (шаг 11)

Запись	Тип	Наш валидатор
`type="xs:string"` / числовой / `xs:anyType`	встроенный, без ограничений	**нарушение**
`type="xs:int"` , `xs:date` , `xs:boolean` …	встроенный, вне правил	**можно**
`type="o:Name"`	свой именованный	проверяется в `Name`
вложенный `simpleType` / `complexType`	анонимный	проверяются его теги
`ref="o:x"`	из глобального `x`	проверяется в `x`
без `type` , с `substitutionGroup`	от головы группы	**можно**
ничего (элемент / атрибут)	неявный `anyType` / `anySimpleType`	**нарушение**

Повторное использование и файлы (шаги 11, 13)

`<xs:element ref="o:note"/>`	вставить глобальный элемент целиком
`<xs:group ref="o:G"/>`	вставить именованный кусок модели
`<xs:attributeGroup ref="o:A"/>`	вставить набор атрибутов
`substitutionGroup="o:head"`	элемент может стоять на месте `head` ; тип — производный от типа головы
`<xs:include schemaLocation="..."/>`	файл с **тем же** `targetNamespace`
`<xs:import namespace="..." schemaLocation="..."/>`	файл с **другим** пространством имён; плюс `xmlns:префикс` для ссылок на его типы

Чек-лист наших правил

Нарушение	Как правильно
`type="xs:string"`	`type="o:Text"` , где `Text` — restriction с `maxLength` / `length` / `pattern` / `enumeration`
`restriction base="xs:string"` только с `minLength` или `whiteSpace`	добавить `maxLength` (или другой засчитываемый фасет)
`extension base="xs:string"` в `simpleContent`	`extension base="o:Text"`
`list itemType="xs:string"` , `union memberTypes="… xs:string"`	указать свой ограниченный тип
`type="xs:decimal"` и другие пять	свой тип с `totalDigits` или min + max
только `fractionDigits` или только одна граница	добавить `totalDigits` или вторую границу
`<xs:any/>`	явно перечислить элементы
`type="xs:anyType"` / `type="xs:anySimpleType"`	конкретный тип
`<xs:element name="x"/>` , `<xs:attribute name="y"/>` без типа	указать `type` или вложенный тип
`complexContent extension base="xs:anyType"`	обычный `complexType` (или `restriction base="xs:anyType"`)

Частые сообщения валидатора

Проверка XML (`Validator`)	
`cvc-elt.1.a`	элемент не объявлен: опечатка или не то пространство имён
`cvc-datatype-valid`	значение не подходит под встроенный тип
`cvc-pattern-valid`	не совпал шаблон
`cvc-enumeration-valid`	нет в списке значений
`cvc-length-valid`, `cvc-min/maxLength-valid`	длина
`cvc-min/maxInclusive-valid`, `…Exclusive-valid`	границы числа
`cvc-totalDigits-valid`, `cvc-fractionDigits-valid`	количество цифр
`cvc-complex-type.2.4.a`	не тот элемент на этом месте (порядок)
`cvc-complex-type.2.4.b`	не хватает обязательного элемента
`cvc-complex-type.2.4.d`	лишний элемент
`cvc-complex-type.3.2.2`	лишний атрибут
`cvc-complex-type.4`	нет обязательного атрибута
`cvc-elt.2`	использован абстрактный элемент

Компиляция схемы (этап 1)	
`s4s-elt-…`	неизвестный или не на своём месте тег схемы
`s4s-att-…`	неизвестный или недостающий атрибут тега схемы
`src-resolve`	не найден тип или элемент: опечатка или нет префикса `o:`
`cos-applicable-facets`	фасет не подходит к типу
`…-valid-restriction`	наследник расширяет, а не сужает
`src-element.3`	и `type`, и вложенный тип
`schema_reference`	не найден подключаемый файл

У ошибок значения Xerces обычно выдаёт две строки: конкретную (`cvc-pattern-valid` …) и общую: `cvc-type.3.1.3` для элемента или `cvc-attribute.3` для атрибута («значение недопустимо»). Смысл несёт первая.